资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

C语言复习大纲

刘子晗

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

复习讲座大纲：C语言课程

☆ 机考考试范围：基本语法、I/O、分支，循环和函数(不涉及数组等地址型数据)。
☆ 题型：
① 两道编程题（每题5分，共10分）
② 一道程序改错题（5分）
③ 五道选择题（每题1分，共5分）
☆ 同一时段试卷分A、B两卷；
☆ 考试时间90分钟。

环境配置

磨刀不误砍柴工

VSCode

vscch

Clion

Github 下载加速

Visual Studio

哈尔滨工业大学
薪火笔记社

关于课程

多写，多练，多看

遇到问题学会搜索，独立解决问题也是能力增长的过程

Lab的Writeup要好好写，尽量别用MS Word排版

最后大家分数不会相差太多，前提是你花时间，~~哪怕是抄~~

期中考试是机考，几乎无限制

注意边界问题 循环嵌套

边边角角的知识点，善用编译器

期末考试是笔试，手写代码，人脑编译，手动运行

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

老师纸质阅卷，字写得工整些，代码逻辑要清晰 → 先法报和复杂判断逻辑

[CS50 Lecture 1 - C]

[CS50 2024]

[CSDIY]

数据类型

基本数据类型（`int`，`float`，`char`，等）

- int 整数类型 4bytes = 32bits `int a = 1;`
- char 存储单个字符/ASCII值 1byte = 8bits `char grade = 'A';`
- double 存储双精度浮点数 8bytes = 64bits 1bit符号位，11bits指数部分，52bits小数部分
- float 存储单精度浮点数 4bytes = 32bits 1bit符号位，8bits指数部分，23bits小数部分
- 浮点数存在误差

哈尔滨工业大学
薪火笔记社

```c
#include <stdio.h>

int main() {
    int i;
    float f_sum = 0.0f;
    double d_sum = 0.0;
    int iterations = 1000000; // 累加次数

    // 每次累加一个很小的值 0.000001
    for (i = 0; i < iterations; i++) {
        f_sum += 0.000001f;  // float 累加
        d_sum += 0.000001;   // double 累加
    }

    printf("After %d iterations:\n", iterations);
    printf("Float sum: %.10f\n", f_sum);
    printf("Double sum: %.10lf\n", d_sum);

    // 真实值应为 iterations * 0.000001，即 1.0
    printf("Expected sum: 1.0000000000\n");

    return 0;
}
```

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

```
After 1000000 iterations:
Float sum: 1.0090389252
Double sum: 1.0000000000
Expected sum: 1.0000000000
```

数据类型	字节数（典型）	有效位数	取值范围	说明
`char`	1 字节	8 位	-128 到 127（`signed`）或 0 到 255（`unsigned`）	存储单个字符或小整数
`short`	2 字节	16 位	-32,768 到 32,767	较小范围的整数
`int`	4 字节	32 位	-2,147,483,648 到 2,147,483,647	常用的整数类型
`long`	4 或 8 字节	32 或 64 位	根据系统，通常与 `int` 或 `long long` 相同	大范围的整数
`long long`	8 字节	64 位	-9,223,372,036,854,775,808 到 9,223,372,036,854,775,807	大范围的整数类型
`float`	4 字节	约 6-7 位有效数字	约 3.4E-38 到 3.4E+38	单精度浮点数
`double`	8 字节	约 15-16 位有效数字	约 1.7E-308 到 1.7E+308	双精度浮点数
`long double`	16 字节（通常）	约 18-19 位有效数字	约 3.4E-4932 到 1.1E+4932	扩展精度浮点数
`Bool`	1 字节	1 位	0 或 1	用于表示布尔值（`true` 或 `false`）
`指针 *`	4 字节(32位) 8 字节(64位)	取决于系统	-	记录地址

哈尔滨工业大学
薪火笔记社

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

常量与变量

常量

程序运行过程中不会改变的量，通常用 `const` 关键字或 `#define` 宏定义来声明。

其优点在于：提升可读性；增加安全性；提高维护性。→ 一键修改

一眼看出 产品手线进行修改

是要避免 Magic Num

```
const int max = 100;
#define PI 3.14
```

变量

程序运行过程中会改变的值，每个变量占据一个特定的内存空间，其值随时更新

```
int a = 1;
double len = 3.14;
char grade = 'A';
int *ptr;
```

哈尔滨工业大学
薪火笔记社

局部变量&全局变量

```
#include <stdio.h>

void func() {
    int x = 10; // 局部变量 x
    printf("Inside func, x = %d\n", x);
}

int main() {
    int x = 5; // 主函数中的局部变量 x
    printf("Inside main, x = %d\n", x);
    func();
    return 0;
}
```

```
#include <stdio.h>

int x = 20; // 全局变量 x

void func() {
    printf("Inside func, x = %d\n", x);
}
```

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

```c
int main() {
    printf("Inside main, x = %d\n", x);
    func();
    return 0;
}
```

特性	局部变量	全局变量
定义位置	定义在函数或代码块内部	定义在所有函数外部
作用域	仅在定义它的函数或代码块内部有效	整个程序文件内有效，跨所有函数
生命周期	在函数或代码块调用时创建，结束时销毁	程序运行时创建，直到程序结束才销毁
存储位置	通常保存在栈（stack）中	通常保存在数据段（data segment）中
命名冲突	不同函数中的局部变量可以同名，互不影响	全局变量在文件中只能定义一次，避免重复定义
变量遮蔽	局部变量可以遮蔽同名的全局变量	全局变量被局部变量遮蔽时无法在局部作用域中访问
适用场景	用于函数内部的临时数据或局部性强的变量	用于需要在多个函数中共享的数据
优先级	优先级高于同名的全局变量	优先级低于同名的局部变量
安全性	更安全，影响范围小，易于管理	易引起意外修改，影响范围广，需谨慎使用

哈尔滨工业大学
薪火笔记社

类型转换

隐式转换

隐式转换是由编译器自动进行的类型转换，不需要显式指定转换方式。这种转换通常发生在不同类型的变量参与同一运算或赋值时。编译器会根据操作数的类型和精度，自动进行类型提升（通常是从低精度到高精度），从而避免数据丢失。

常见的隐式转换规则

- **从低精度到高精度**：在混合运算中，编译器会将低精度的类型（如 `int` ）转换为高精度类型（如 `float` 或 `double` ）。
- **整型提升**：在表达式中， `char` 和 `short` 会自动提升为 `int` 类型，以保证运算精度。
- **浮点数提升**： `float` 类型在与 `double` 类型运算时会提升为 `double` 。

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

示例

```
#include <stdio.h>

int main() {
    int a = 5;
    float b = 4.5;
    float result = a + b; // 隐式转换，将 a 从 int 转换为 float

    printf("Result: %f\n", result); // 输出 9.5
    return 0;
}
```

说明：在 `a + b` 的运算中，`a`（`int` 类型）会被自动转换为 `float`，从而与 `b` 一起参与 `float` 运算。

显式转换

显式转换，也称为**强制类型转换**，是程序员通过类型转换运算符显式地将一种数据类型转换为另一种数据类型。显式转换可以解决隐式转换无法满足的需求，但使用时需要特别小心，因为可能导致数据丢失。

哈尔滨工业大学
薪火笔记社

显式转换的语法

```
(目标类型) 表达式
```

示例

```
#include <stdio.h>

int main() {
    int a = 5, b = 2;
    float result;

    result = (float)a / b; // 强制将 a 转换为 float 类型，再与 b 相除

    printf("Result: %f\n", result); // 输出 2.500000
    return 0;
}
```

说明：在 `(float)a / b` 中，`a` 被强制转换为 `float`，这样除法结果也是 `float` 类型，否则结果会因为整数除法而丢失小数部分。

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

类型转换的注意事项

- **数据丢失**：类型转换可能导致数据丢失。例如，将 `double` 转换为 `int` 时会丢失小数部分，将 `int` 转换为 `char` 时可能会超出 `char` 的存储范围。
- **精度问题**：浮点数转换为整数会丢失小数部分，同时注意浮点数的精度限制；`float` 转换为 `double` 不会丢失数据，但会影响运算速度。
- **避免不必要的隐式转换**：如果不明确指定数据类型，编译器会根据规则进行自动转换，有时可能会引入隐式错误。例如在 `int` 和 `double` 混合运算时，最好**使用显式类型**转换来确保预期结果。

示例：类型转换导致的错误

```c
#include <stdio.h>

int main() {
    int x = 10;
    int y = 3;
    float result = x / y; // x 和 y 都是 int 类型，结果会进行整数除法

    printf("Result: %f\n", result); // 输出 3.000000，而非 3.333333
    return 0;
}
```

哈尔滨工业大学
薪火笔记社

说明：由于 `x / y` 都是 `int`，整数除法导致小数部分丢失。正确的做法是将其中一个变量强制转换为 `float` 类型，如 `result = (float)x / y;`，这样就可以得到浮点数结果 `3.333333`。

输入输出

1. 标准输入输出函数（`printf` 和 `scanf`）

- `printf` **函数**：用于将输出结果显示在屏幕上。可以输出字符串、整数、浮点数等多种类型。
  - 基本语法：`printf("格式字符串", 参数1, 参数2, ...);`
  - 示例：

```c
int age = 25;
printf("Your age is %d\n", age);
```

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

  - **返回值**：`printf` 返回成功打印的字符数。
  - **常见错误**：格式字符串和参数不匹配会导致未定义行为或编译错误。
- `scanf` **函数**：用于从标准输入读取数据（通常是键盘输入）。
  - 基本语法：`scanf("格式字符串", &变量1, &变量2, ...);`
  - 示例：

```
int age = 0;
printf("Enter your age: "); // 提示信息单独打印
scanf("%d", &age);
```

  - **注意事项**：`scanf` 需要传递变量的地址（用 `&` 符号），以便直接修改变量的值。
  - **返回值**：`scanf` 返回成功读取的变量个数，通常可以用于判断输入是否成功。

2. 格式控制符的使用

控制符	数据类型	说明
`%d`	`int`	带符号的十进制整数
`%u`	`unsigned int`	无符号十进制整数
`%ld`	`long int`	带符号的长整数（long）
`%lu`	`unsigned long int`	无符号长整数
`%lld`	`long long int`	带符号的长长整数（long long）
`%llu`	`unsigned long long int`	无符号长长整数（long long）
`%x` / `%X`	`unsigned int`	无符号十六进制整数（`%x` 小写，`%X` 大写）
`%o`	`unsigned int`	无符号八进制整数
`%c`	`char`	单个字符
`%s`	`char*`	字符串，自动输出到遇到 `\0` 结束
`%f`	`float` / `double`	浮点数（默认小数点后保留 6 位）
`%lf`	`double`	双精度浮点数（`double`）
`%Lf`	`long double`	扩展精度浮点数（`long double`）
`%e` / `%E`	`float` / `double`	科学计数法表示的浮点数（`%e` 小写，`%E` 大写）
`%g` / `%G`	`float` / `double`	根据数值自动选择 `%f` 或 `%e` 格式（`%g` 小写，`%G` 大写）
`%%`		输出 `%` 符号
`%p`	指针地址	显示指针的地址值

哈尔滨工业大学
薪火笔记社

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

转义字符表

转义字符	含义	说明
`\n`	换行符	光标移动到下一行开头
`\t`	水平制表符（Tab）	插入水平制表符，使光标移动到下一个制表位置
`\\`	反斜杠	输出 `\` 符号
`\'`	单引号	输出单引号 `'`，常用于字符数据
`\"`	双引号	输出双引号 `"`，常用于字符串数据
`\a`	响铃	触发系统的蜂鸣或响铃（若系统支持）
`\b`	退格符	将光标向左移动一位
`\r`	回车符	将光标移到当前行的开头
`\f`	换页符	换页或清屏
`\v`	垂直制表符	插入垂直制表符，使光标移动到下一个垂直制表位置

示例

哈尔滨工业大学
薪火笔记社

```c
int age = 25;
float height = 175.5;
printf("Age: %d, Height: %.2f\n", age, height);  // 输出: Age: 25, Height: 175.50
```

3. 处理输入输出中的常见问题

- **格式不匹配**：如果格式控制符和变量类型不匹配，可能会导致错误或意外行为。例如，`scanf("%d", &f);`（`f` 是 `float` 类型）会导致未定义行为。
- **输入缓冲区问题**：`scanf` 会将回车键等保存在输入缓冲区中，可能影响后续的输入。例如，读取字符时经常需要处理换行符。
  - 解决方法：可以使用 `getchar()` 清空输入缓冲区。

```c
#include <stdio.h>

int main() {
    int num;
    char ch;

    printf("Enter an integer: ");
    scanf("%d", &num);  // 输入整数
```

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

```c
    printf("Enter a character: ");
    ch = getchar();      // 尝试读取一个字符

    printf("You entered integer: %d and character: '%c'\n", num, ch);
    return 0;
}
```

```
Enter an integer:12
 12
Enter a character: You entered integer: 12 and character: '
'
```

```c
#include <stdio.h>

int main() {
    int num;
    char ch;

    printf("Enter an integer: ");
    scanf("%d", &num);  // 输入整数

    getchar();  // 清除缓冲区中的换行符

    printf("Enter a character: ");
    ch = getchar();

    printf("You entered integer: %d and character: '%c'\n", num, ch);
    return 0;
}
```

哈尔滨工业大学
薪火笔记社

```
Enter an integer:12
 12
Enter a character:c
 c
You entered integer: 12 and character: 'c'
```

- **数据范围溢出**：如果输入超出变量的存储范围（例如将 `int` 型变量赋值超过其最大值），会导致溢出。

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

- **安全输入**：使用 `scanf("%s", str);` 读入字符串时存在**缓冲区溢出**的风险，建议限制最大输入长度，例如 `scanf("%19s", str);` 限制字符串最大长度为19。

4. 文件输入输出（`fopen`，`fread`，`fwrite` 等）

文件输入输出允许程序从文件中读取数据或将数据写入文件，是数据持久化的一种方式。

- **文件打开**（`fopen`）：
  - `fopen` 用于打开文件，返回文件指针，模式包括 `"r"`（只读）、`"w"`（写入）、`"a"`（追加）等。
  - 示例：`FILE *fp = fopen("data.txt", "r");`
- **读取文件**（`fread`，`fgets`，`fscanf`）：
  - `fread`：用于从文件读取二进制数据，通常适用于结构体等复杂数据类型。
  - `fgets`：从文件中读取一行文本，适用于文本文件。
  - `fscanf`：与 `scanf` 类似，但从文件中读取格式化数据。
  - `fread(void *ptr, size_t size, size_t count, FILE *stream);`
    - ptr 指向数据存储位置的指针
    - size 每个元素的字节大小
    - count 要读取的元素数量
    - stream 文件指针
    - 返回值：成功读取的元素数量
    -

```c
#include <stdio.h>

struct Data {
    int id;
    float value;
};

int main() {
    FILE *fp = fopen("data.bin", "rb"); // 打开文件，读取二进制数据
    struct Data d;

    if (fp != NULL) {
        fread(&d, sizeof(struct Data), 1, fp); // 从文件读取一个 Data 结构
        printf("ID: %d, Value: %f\n", d.id, d.value);
        fclose(fp);
    } else {
        printf("Failed to open file.\n");
    }
```

哈尔滨工业大学
薪火笔记社

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

```
    return 0;
}
```

- `fgets(char *str, int n, FILE *stream);`
  - str 指向字符串的指针，用于存储数据
  - n 最多读取的字符数，读到 n-1 个字符或遇到 `\n` 时停止读取
  - stream 文件指针，指向需要读取的文件
  - 返回值：成功返回 `str`，错误返回 `NULL`
  -
    ```
    #include <stdio.h>

    int main() {
        FILE *fp = fopen("example.txt", "r"); // 打开文件，读取文本数据
        char buffer[100];

        if (fp != NULL) {
            if (fgets(buffer, 100, fp) != NULL) { // 从文件读取一行
                printf("Read line: %s", buffer);
            }
            fclose(fp);
        } else {
            printf("Failed to open file.\n");
        }

        return 0;
    }
    ```

哈尔滨工业大学
薪火笔记社

- `fscanf(FILE *stream, const char *format, ...);`
  - stream 文件指针，指向需要读取的文件
  - format 格式字符串，指定要读取的文件格式，例如 `%d %s %f`
  - ... 格式化字符串对应的变量地址，用于存储读取到的数据
  - 返回值：成功读取的字段数量
  -
    ```
    #include <stdio.h>

    int main() {
        FILE *fp = fopen("data.txt", "r");
        int id;
        float value;

        if (fp != NULL) {
            fscanf(fp, "%d %f", &id, &value); // 从文件中读取格式化数据
    ```

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

```
        printf("ID: %d, Value: %.2f\n", id, value);
        fclose(fp);
    } else {
        printf("Failed to open file.\n");
    }

    return 0;
}
```

- **写入文件**（`fwrite`，`fprintf`，`fputs`）：
  - `fwrite`：用于写入二进制数据，适合写入结构体或二进制数据。
  - `fprintf`：类似 `printf`，将格式化输出写入文件。
  - `fputs`：将字符串写入文件，不会自动换行。
  - 示例：

```
fprintf(fp, "Name: %s, Age: %d\n", name, age);
```

- **关闭文件**（`fclose`）：
  - 文件操作完成后，使用 `fclose(fp);` 关闭文件，释放资源。

哈尔滨工业大学
薪火笔记社

示例代码

```
#include <stdio.h>

int main() {
    FILE *fp = fopen("data.txt", "w"); // 打开文件进行写入
    if (fp != NULL) {
        fprintf(fp, "Hello, world!\n"); // 写入数据
        fclose(fp); // 关闭文件
    }

    fp = fopen("data.txt", "r"); // 打开文件进行读取
    if (fp != NULL) {
        char buffer[100];
        fgets(buffer, 100, fp); // 从文件读取一行数据
        printf("File content: %s", buffer);
        fclose(fp); // 关闭文件
    }

    return 0;
}
```

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

选择结构

1. 条件判断语句

条件判断语句允许程序根据表达式的真假来决定执行哪个代码块。最常见的条件判断语句有 `if` 和 `if-else` 。

1.1 `if` 语句

`if` 语句是最基本的条件判断，它根据给定的条件来决定是否执行特定代码。

```
if (condition) {
    // 当条件为真时执行的代码
}
```

例如：

```
int score = 85;
if (score ≥ 60) {
    printf("Passed.\n");
}
```

哈尔滨工业大学
薪火笔记社

在上述例子中，如果 `score` 的值大于或等于 60，则输出 "Passed."。

1.2 `if-else` 语句

`if-else` 语句用于处理两个互斥的分支，即当 `if` 条件不成立时执行 `else` 部分的代码。

```
if (condition) {
    // 当条件为真时执行的代码
} else {
    // 当条件为假时执行的代码
}
```

例如：

```
int score = 45;
if (score ≥ 60) {
    printf("Passed.\n");
} else {
    printf("Failed.\n");
}
```

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

在上述例子中，如果 `score` 小于 60，则输出 "Failed."。

2. 多分支选择：`switch` 语句

`switch` 语句用于处理多分支情况，是多个 `if-else` 语句的简洁替代方案。`switch` 语句特别适合对一个变量进行多种可能值的判断。

```
switch (variable) {
    case value1:
        // 当变量的值为 value1 时执行的代码
        break;
    case value2:
        // 当变量的值为 value2 时执行的代码
        break;
    // 可以有多个 case
    default:
        // 当没有匹配的 case 时执行的代码
        break;
}
```

哈尔滨工业大学
薪火笔记社

例如：

```
char grade = 'B';
switch (grade) {
    case 'A':
        printf("Excellent!\n");
        break;
    case 'B':
        printf("Good job.\n");
        break;
    case 'C':
        printf("Well done.\n");
        break;
    default:
        printf("Invalid grade.\n");
        break;
}
```

在上述例子中，根据 `grade` 的不同值执行不同的代码分支。

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

2.1 `switch` 的使用场景

- 当需要根据一个变量的值执行多个不同分支时，`switch` 语句比多个 `if-else` 更为简洁、可读。
- 适用于处理有限个可能取值的情形，如菜单选项、状态机等。

3. 嵌套条件与逻辑运算符

有时我们需要在一个 `if` 或 `switch` 中嵌套另一个 `if` 或 `switch`，这被称为嵌套条件。通过嵌套条件，程序可以处理更复杂的判断逻辑。

3.1 嵌套 `if` 语句

```
int age = 20;
int score = 75;

if (age ≥ 18) {
    if (score ≥ 60) {
        printf("Adult Passed.\n");
    } else {
        printf("Adult Failed.\n");
    }
} else {
    printf("Not an adult.\n");
}
```

哈尔滨工业大学
薪火笔记社

在上述例子中，只有当 `age` 大于等于 18 时，才会进一步判断 `score` 是否大于等于 60。

3.2 逻辑运算符

逻辑运算符用于组合多个条件判断，以形成更为复杂的逻辑。常用的逻辑运算符有：

- **逻辑与 (`&&`)**：所有条件都为真时，表达式结果为真。
- **逻辑或 (`||`)**：只要有一个条件为真，表达式结果为真。
- **逻辑非 (`!`)**：取反，将条件的真假性取反。

例如：

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

```c
int age = 20;
int score = 75;

if (age ≥ 18 && score ≥ 60) {
    printf("Adult Passed.\n");
} else {
    printf("Adult Failed or Not an Adult.\n");
}
```

在上述例子中，只有当 `age` 大于等于 18 且 `score` 大于等于 60 时，才会输出 "Adult Passed."。

循环结构

1. `for`、`while` 和 `do-while` 循环

循环结构在编程中用于重复执行某一段代码，`for`、`while` 和 `do-while` 是 C 语言中常用的循环结构。

哈尔滨工业大学
薪火笔记社

1.1 `for` 循环

`for` 循环通常用于已知循环次数的场景。其语法形式如下：

```c
for (初始化表达式; 条件表达式; 更新表达式) {
    // 循环体
}
```

- **初始化表达式**：在循环开始前执行，用于初始化循环变量。
- **条件表达式**：每次循环前进行判断，只有条件为 `true` 时，循环体才会执行。
- **更新表达式**：每次执行完循环体后执行，用于更新循环变量。

示例：

```c
for (int i = 0; i < 10; i++) {
    printf("%d\n", i);
}
```

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

1.2 `while` 循环

`while` 循环通常用于循环次数不确定的场景。其语法形式如下：

```
while (条件表达式) {
    // 循环体
}
```

- **条件表达式**：在每次执行循环体前进行判断，只有条件为 `true` 时，循环体才会执行。
- 如果循环开始时条件表达式为 `false`，则循环体一次都不会执行。

示例：

```
int i = 0;
while (i < 10) {
    printf("%d\n", i);
    i++;
}
```

哈尔滨工业大学
薪火笔记社

1.3 `do-while` 循环

`do-while` 循环用于至少需要执行一次循环体的场景。其语法形式如下：

```
do {
    // 循环体
} while (条件表达式);
```

- **循环体**：在每次执行条件判断之前执行一次，因此无论条件表达式是否为 `true`，循环体至少会执行一次。

示例：

```
int i = 0;
do {
    printf("%d\n", i);
    i++;
} while (i < 10);
```

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

1.4 使用与区别总结

- `for` 循环：适合循环次数已知的场景。
- `while` 循环：适合循环次数不确定，但需要在进入循环前判断条件的场景。
- `do-while` 循环：适合至少要执行一次循环体的场景。

2. 循环控制语句：`break` 和 `continue`

循环控制语句用于改变循环的正常执行流程，`break` 和 `continue` 是最常用的两种控制语句。

2.1 `break` 语句

`break` 语句用于立即终止所在的循环或 `switch` 语句，并跳出循环体。它通常用于某个特定条件满足时结束循环。

示例：

```
for (int i = 0; i < 10; i++) {
    if (i == 5) {
        break;
    }
    printf("%d\n", i);
}
```

哈尔滨工业大学
薪火笔记社

当 `i` 等于 5 时，`break` 语句会终止循环，输出结果为 0 到 4。

2.2 `continue` 语句

`continue` 语句用于跳过本次循环剩余的语句，并开始下一次循环判断。它常用于需要跳过某些特定条件下的操作而继续执行其他循环的场景。

示例：

```
for (int i = 0; i < 10; i++) {
    if (i == 5) {
        continue;
    }
    printf("%d\n", i);
}
```

当 `i` 等于 5 时，`continue` 语句会跳过当次循环，输出结果为 0 到 4，6 到 9。

资料免费开源，欢迎关注公众号薪火笔记社，加入我们的开源计划！
资料版权由薪火笔记社和作者共同所有，禁止盗用及倒卖！

3. 嵌套循环及其应用场景

嵌套循环是指在一个循环体内再包含一个或多个循环。嵌套循环通常用于处理二维或多维数据，例如矩阵运算、图形绘制等。

3.1 嵌套循环的基本形式

```
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        // 内层循环体
    }
}
```

在嵌套循环中，外层循环每执行一次，内层循环会执行一遍完整的循环。

3.2 应用场景示例

1. 二维数组的遍历

嵌套循环通常用于遍历二维数组的每一个元素。

```
int matrix[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        printf("%d ", matrix[i][j]);
    }
    printf("\n");
}
```

哈尔滨工业大学
薪火笔记社

2. 打印图形

嵌套循环可以用于打印各种图形，例如打印一个矩形或三角形。

```
for (int i = 0; i < 5; i++) {
    for (int j = 0; j ≤ i; j++) {
        printf("* ");
    }
    printf("\n");
}
```

上述代码会打印一个由星号组成的直角三角形。